Platform Track

Interview Preparation Series • Professional Edition

Observability

Compiled with LaTeX • June 30, 2026

harsha.somisetty01@gmail.com

In This Edition

> **How to use this file:** Read top-to-bottom for deep, mechanism-level understanding of how telemetry actually works. Jump to §Common Interview Questions + §Revision Cheat Sheet for last-minute revision. A system design isn't finished until you can *see* it running — metrics, logs, traces, and SLOs come up in **every** senior/staff design round, and "your service is on fire, debug it" is a standard interview prompt. This file teaches you to answer both.

1 Overview --- What It Is

Observability is the ability to understand a system's internal state purely from its external outputs — to ask *new* questions about *why* it's behaving a certain way **without shipping new code**. The term comes from control theory: a system is "observable" if you can reconstruct its internal state from its outputs. Applied to software, it means your telemetry is rich enough that when something unexpected breaks, you can investigate it with the data you already emit.

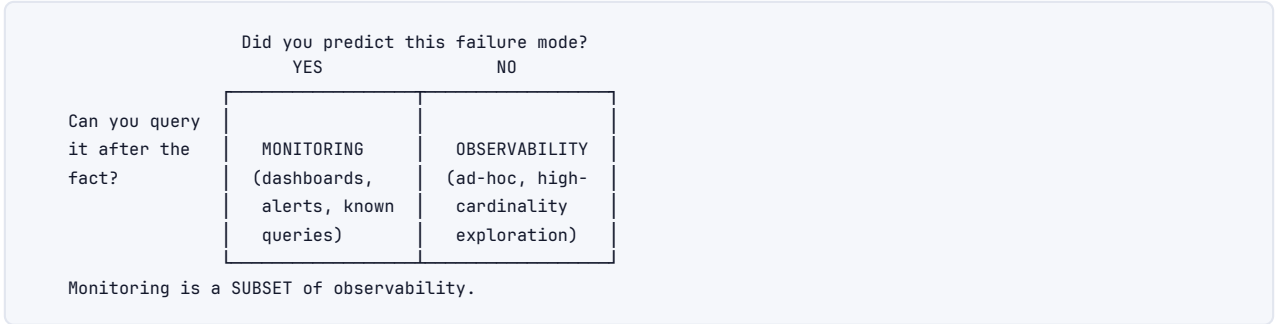
It is built on **three pillars** — metrics, logs, and traces — plus the practices that turn that data into reliable operations: SLOs, alerting, and incident response.

```
Metrics → what is happening   (aggregate numbers, trends, cheap)
Logs    → what happened       (discrete events, full detail, expensive)
Traces  → where it happened   (one request's path across services)
```

The mental model: **metrics tell you something is wrong, traces tell you where, logs tell you why.** A good observability practice links all three (by a shared trace id) so you can pivot seamlessly: alert fires on a metric → open the trace for an exemplar slow request → read the structured logs for that exact request.

The crucial distinction — Monitoring vs Observability:

- **Monitoring** watches **known** failure modes. You decide in advance what could go wrong ("CPU might spike," "the queue might back up," "error rate might rise") and build dashboards and alerts for each. Monitoring answers *known-knowns* and *known-unknowns*: "Is the thing I anticipated broken?"
- **Observability** is the *property* that lets you debug **unknown-unknowns** — failures you never predicted. "Why is p99 latency up only for Android users in the EU region on the `/checkout` endpoint when they pay with PayPal?" You never built a dashboard for that exact slice — but if your telemetry is high-cardinality and correlatable, you can discover it on the fly.



Interview takeaway: Monitoring tells you *that* the known thing broke. Observability lets you ask *why* the unknown thing broke — without deploying. Saying "monitoring is a subset of observability" signals you understand the difference instead of using them as synonyms.

2 Why It Exists

2.1 The distributed-systems problem

In a monolith on one box, debugging was `ssh` + `tail -f app.log` + `top`. You could see everything because everything was in one place. Distributed systems destroyed that:

- A single user request fans out across **dozens** of microservices, each on different hosts, in different teams' code, written in different languages.
- When a request is slow or fails, "**which hop, for which users, why?**" is unanswerable by staring at one machine.
- Failures are **partial and emergent**: service A is fine, B is fine, but A→B under load with a specific payload times out. No single component is "down."
- Hosts are **ephemeral** (autoscaling, K8s pods come and go), so "ssh into the box" doesn't work — the box may already be gone.

Observability exists to reconstruct a coherent picture of the system from telemetry emitted by hundreds of moving parts.

2.2 What it buys you

1. **Cuts MTTR (Mean Time To Recovery).** The difference between a 5-minute incident and a 5-hour incident is almost always telemetry. If you can jump from "errors are up" to "this one downstream dependency is timing out" in 30 seconds, you mitigate fast.
2. **Surfaces unknown-unknowns.** Emergent failures you never anticipated become discoverable.
3. **Closes the reliability loop.** Measure → set SLOs → alert on burn → improve. Without measurement there is no objective notion of "reliable enough."
4. **Quantifies the velocity/reliability trade-off.** Error budgets turn "are we reliable enough to ship?" from an argument into arithmetic.

> You cannot operate what you cannot see. At scale, observability is not optional infrastructure — it is *how you keep the lights on*, and it must be designed in, not bolted on.

3 Why FAANG Cares

Company	Evidence / Origin	Lesson
Google	Coined **SRE**, **SLI/SLO/error budget** in the SRE book. Built **Borgmon** (Prometheus' ancestor), **Dapper** (the paper that defined distributed tracing — spans, trace context propagation).	The vocabulary of modern observability is Google's. Expect deep SLO/error-budget questions.
Amazon	"Operational excellence" is the **first pillar** of the AWS Well-Architected Framework. Famous internal rule: *you build it, you run it* — devs carry the pager.	If you build it you must observe it. Designs are judged on operability.
Meta	Built **Scuba** (real-time high-cardinality analytics) and a culture of slicing telemetry by arbitrary dimensions at query time.	High-cardinality, ad-hoc querying is the future of debugging.
Netflix	Pioneered **chaos engineering** (you must observe the blast radius), **Atlas** (in-house metrics), per-second streaming SLIs.	You can't do chaos engineering without observability to measure the damage.

Company	Evidence / Origin	Lesson
Honeycomb (ex-Facebook founders)	Popularized "**observability**" as distinct from monitoring; built around high-cardinality events and `BubbleUp` exploratory analysis.	The industry's modern definition of observability comes from here.

FAANG interview reality: Two guaranteed touchpoints. (1) In *every* system design, the interviewer will ask "**how would you monitor and debug this in production?**" — a vague answer signals a junior. (2) A debugging round: "**p99 latency tripled, walk me through how you'd find the cause.**" They want a disciplined metrics→traces→logs workflow, not guessing.

Interview takeaway: End every system design with an explicit observability section — key SLIs, an SLO with an error budget, the three pillars linked by a trace id, and symptom-based alerts. This is the single highest-leverage thing you can add to a design answer.

4 Core Concepts

Before diving into each pillar, lock down the vocabulary. These terms recur everywhere and interviewers use them as shibboleths.

Concept	Definition	Why it matters
Telemetry	The data a system emits about itself (metrics, logs, traces, profiles).	The raw material of observability.
Cardinality	The number of *unique* values a dimension/label can take, and by extension the number of unique time-series produced.	The #1 cost and scalability driver. High cardinality (per-user labels) explodes storage.
Dimension / Label / Tag	A key-value attribute attached to telemetry (`region="eu"`, `endpoint="/checkout"`).	More dimensions = more ways to slice = more cardinality.
Aggregation	Rolling many raw data points into a summary (sum, rate, percentile) over a time window.	Metrics are pre-aggregated and cheap; logs are raw and expensive.
Sampling	Keeping only a fraction of traces/logs to control cost.	The lever for affordable tracing at scale.
Time-series	A stream of `(timestamp, value)` pairs identified by a metric name + label set.	The fundamental unit of metrics storage.
Exemplar	A pointer from an aggregated metric bucket to a specific trace that landed in it.	The bridge that links a metrics spike directly to a trace.
OpenTelemetry (OTel)	The vendor-neutral CNCF standard: APIs, SDKs, and the Collector for emitting all three pillars.	Instrument once, export anywhere. The default answer to "how do you instrument?"
MTTD / MTTR / MTBF	Mean Time To Detect / Recover / Between Failures.	The headline numbers observability and incident response improve.

4.1 High-cardinality querying --- the heart of observability

The thing that separates "monitoring" from "observability" is the ability to slice telemetry by **high-cardinality dimensions at query time**. Consider:

- `user_id` — millions of distinct values
- `request_id` — unbounded
- `build_id`, `feature_flag`, `device_model`, `app_version` — hundreds to thousands each

A traditional metrics system *cannot* store a separate time-series per `user_id` (more on the explosion math below). But an **event-based** store (Honeycomb, Scuba, or a wide structured log/trace store) keeps the raw high-cardinality events and lets you `GROUP BY user_id` after the fact. That's how you discover "it's only failing for these 3 enterprise customers on app version 4.2.1" — a query nobody anticipated.

Interview takeaway: The power of observability is asking questions you didn't pre-plan. That requires high-cardinality, raw, correlatable data — which is fundamentally in tension with the cheap, pre-aggregated, low-cardinality nature of metrics. Knowing that tension (and that you solve it with traces/structured-events plus sampling) is a senior signal.

5 Metrics

A **metric** is a numeric measurement aggregated over time — a time-series identified by a name plus a set of labels. Metrics are **cheap** (pre-aggregated), **fast to query**, and ideal for **dashboards and alerting**. Their weakness is **cardinality**: you cannot attach unbounded labels.

5.1 Metric types --- the four kinds and what each is for

Prometheus (the de-facto open standard) defines four metric types. Understanding *what each is for* and *how percentiles emerge from histograms* is a classic interview probe.

Counter

A value that only ever **goes up** (monotonically increasing), resetting to 0 only on process restart. You never read a counter's raw value directly — you read its **rate**.

- Examples: `http_requests_total`, `errors_total`, `bytes_sent_total`.
- Why monotonic? So that if a scrape is missed, the cumulative value is self-healing — `rate()` reconstructs the per-second increase across any two scrapes and automatically handles counter resets.

```promql
# Requests per second over the last 5 minutes (per series)
rate(http_requests_total[5m])

# Total request rate across all instances of a service
sum(rate(http_requests_total{service="checkout"}[5m]))
```

Gauge

A value that can go **up or down** — a snapshot of "right now."

- Examples: `memory_usage_bytes`, `queue_depth`, `active_connections`, `temperature_celsius`, `goroutines`.
- You read gauges directly (current value), or use functions like `max_over_time`, `delta`, `deriv`.

```promql
# Current queue depth, max over the last hour
max_over_time(queue_depth[1h])
```

Histogram

Samples observations (e.g., request durations) into **predefined buckets**, and exposes a counter per bucket. This is how you compute percentiles **server-side** in a way that aggregates

correctly across instances.

A histogram for `http_request_duration_seconds` with buckets `[0.1, 0.3, 1, 2.5, +Inf]` emits cumulative bucket counters (`_bucket{le="..."}`), a `_sum`, and a `_count`:

```python
http_request_duration_seconds_bucket{le="0.1"}   24054   # ≤100ms
http_request_duration_seconds_bucket{le="0.3"}   33444   # ≤300ms
http_request_duration_seconds_bucket{le="1"}     34522   # ≤1s
http_request_duration_seconds_bucket{le="2.5"}   34588   # ≤2.5s
http_request_duration_seconds_bucket{le="+Inf"}  34601   # all requests
http_request_duration_seconds_sum                 8953.3  # total seconds
http_request_duration_seconds_count               34601   # total requests
```

Note the buckets are **cumulative** (`le` = "less than or equal"): the `le="0.3"` bucket includes everything in `le="0.1"`. This is what makes them aggregatable — you can `sum()` bucket counters across all instances *before* computing the quantile, which is mathematically correct (unlike averaging pre-computed percentiles, which is meaningless).

How a percentile is computed from buckets — `histogram_quantile()` finds which bucket the target rank falls into and **linearly interpolates** within it:

```promql
# p99 latency over 5m, aggregated across all instances
histogram_quantile(
  0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Walk-through for p99 with the numbers above: total count = 34601, so the 99th percentile rank is at observation `0.99 × 34601 ≈ 34255`. That rank lands between the `le="0.3"` bucket (33444) and `le="1"` bucket (34522). Linear interpolation within `[0.3s, 1s]`: `0.3 + (34255 - 33444)/(34522 - 33444) × (1 - 0.3) = 0.3 + (811/1078) × 0.7 ≈ 0.83s`. So **p99 ≈ 830ms**.

The accuracy caveat: histogram percentiles are only as good as your bucket boundaries. If 99% of traffic is between 0.3s and 1s, that's a *wide* interpolation gap — your p99 estimate could be off by hundreds of ms. **Choose buckets around your SLO thresholds** (e.g., if SLO is 200ms, have buckets at 0.1, 0.15, 0.2, 0.25, 0.3) so the percentile near your target is precise.

Summary

Like a histogram, but quantiles are **pre-computed client-side** at scrape time (e.g., the app calculates its own p50/p99 over a sliding window).

```python
http_request_duration_seconds{quantile="0.5"}   0.052
http_request_duration_seconds{quantile="0.99"}  0.84
http_request_duration_seconds_sum                8953.3
http_request_duration_seconds_count              34601
```

- **Pro:** exact quantiles (no bucket interpolation error), cheap to query.
- **Con (fatal at scale):** you **cannot aggregate quantiles across instances**. The p99 of instance A and p99 of instance B cannot be combined into a fleet p99 — there is no mathematical operation to do so. With histograms you `sum` the buckets first, then compute; with summaries you're stuck.

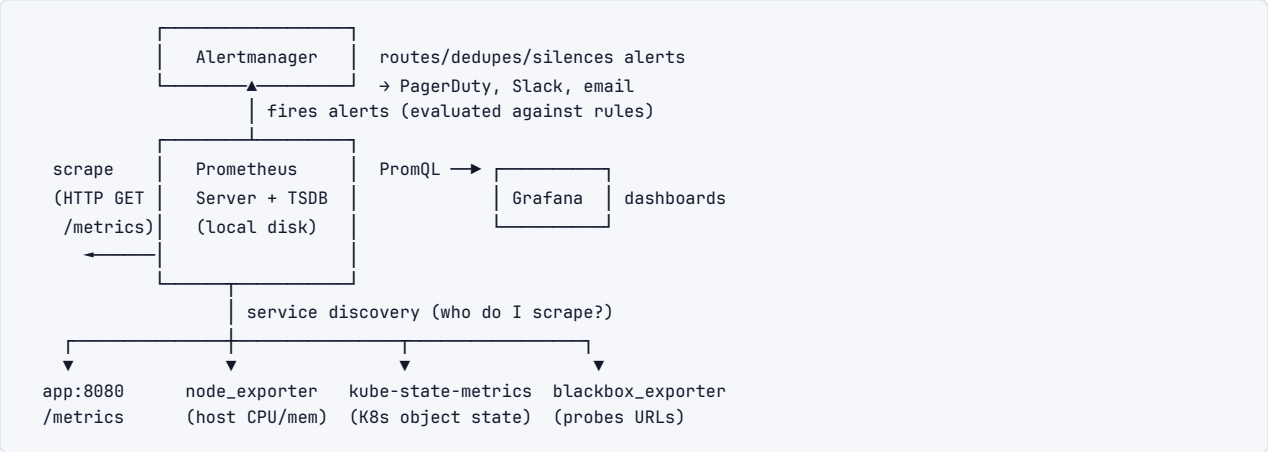
Interview takeaway: Prefer histograms over summaries for anything that runs on more

than one instance, because histogram buckets are aggregatable across instances and summary quantiles are not. The only knob histograms require is choosing bucket boundaries near your SLO. ”You can't average percentiles” is a line that signals you understand the math.

Type	Goes down?	Aggregatable across instances?	Use for
Counter	No	Yes (`sum`)	rates: requests, errors, bytes
Gauge	Yes	Yes (`sum`/`avg`/`max`)	levels: memory, queue depth, connections
Histogram	n/a	**Yes** (sum buckets, then quantile)	latency / size distributions → percentiles
Summary	n/a	**No** (quantiles can't be merged)	single-instance percentiles only

5.2 Prometheus: the pull model, exporters, and service discovery

Prometheus **pulls** (scrapes) metrics from targets over HTTP at a fixed `scrape_interval` (default 15s). Each target exposes a `/metrics` endpoint in a simple text format.



Exporters translate something that *doesn't* speak Prometheus into the `/metrics` format:

- `node_exporter` — host-level CPU, memory, disk, network (the USE method's raw material).
- `kube-state-metrics` — Kubernetes object state (pod restarts, deployment replicas).
- `blackbox_exporter` — probes endpoints from outside (HTTP 200? TLS cert expiry? DNS resolves?).
- `mysqld_exporter`, `redis_exporter`, etc. — pull stats out of off-the-shelf systems.

Service discovery answers ”what should I scrape?” dynamically — critical because targets (pods) are ephemeral. Prometheus integrates with Kubernetes SD, EC2 SD, Consul, etc. It watches the K8s API and automatically scrapes pods that carry a `prometheus.io/scrape: "true"` annotation, so you never hand-maintain a target list.

```yaml
scrape_configs:
  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      # only scrape pods annotated prometheus.io/scrape=true
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
```

```
      # use the annotated port
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port,
                        __address__]
        action: replace
        regex: (\d+);(.+):\d+
        replacement: $2:$1
        target_label: __address__
```

Pull vs Push:

	Pull (Prometheus)	Push (StatsD, Graphite, OTLP-push)
Liveness	Scrape failure *is* a down signal — you know the target is gone	Absence of pushes is ambiguous (down? or just quiet?)
Config	Centralized in Prometheus (SD)	Each app must know where to push
Short-lived jobs	Hard (job ends before scrape) → use **Pushgateway**	Natural fit (batch job pushes on completion)
Network	Prometheus must reach targets (firewall/NAT friction)	Targets reach collector (easier through NAT)
Scale	Sharding/federation needed for huge fleets	Collector can be a bottleneck/SPOF

The **Pushgateway** is the escape hatch for the pull model's weakness: a short-lived batch/cron job that finishes before any scrape pushes its final metrics to the gateway, which Prometheus then scrapes. Caveat: don't use it for long-running services (it breaks the liveness signal and metrics go stale).

Interview takeaway: Pull's killer feature is that a failed scrape is itself a health signal — you instantly know a target is unreachable. The classic pull weakness is short-lived jobs, solved by the Pushgateway. OpenTelemetry's OTLP is push-based, so modern stacks often mix both.

5.3 The three monitoring methodologies --- RED, USE, Four Golden Signals

These tell you *which* metrics to collect. Interviewers love "what would you monitor?" — name the framework.

RED (Tom Wilkie / Weaveworks) — per *service* (request-driven):

- **R**ate — requests per second.
- **E**rrors — failed requests per second (and as a ratio).
- **D**uration — latency distribution (p50/p99 via histograms).

```promql
# RED for the checkout service
# Rate
sum(rate(http_requests_total{service="checkout"}[5m]))
# Errors (ratio)
sum(rate(http_requests_total{service="checkout",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{service="checkout"}[5m]))
# Duration (p99)
histogram_quantile(0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket{service="checkout"}[5m])))
```

USE (Brendan Gregg) — per *resource* (infrastructure-driven):

- **U**tilization — % busy (CPU at 80%).

- **S**aturation — extra queued work the resource can't service yet (run-queue length, disk I/O queue depth).
- **E**rrors — error events (packet drops, ECC errors, disk errors).

Four Golden Signals (Google SRE book): Latency, Traffic, Errors, Saturation.

```
RED  ≈ Latency(Duration) + Traffic(Rate) + Errors   → great for stateless services
USE  ≈ Saturation + Errors + Utilization             → great for resources/queues
Golden Signals = Latency + Traffic + Errors + Saturation = RED ∪ USE
```

Method	Scope	Best for	Misses
RED	request-driven services	microservices, APIs	saturation/resource limits
USE	resources (CPU, disk, queues)	hosts, databases, message queues	user-facing experience
Golden Signals	services holistically	the union; the default	(the complete set)

A subtle point — latency of errors: when computing Duration, measure successful and failed requests **separately**. A flood of fast 500s (failing instantly) can *lower* your average latency and hide a problem. Always split error latency from success latency.

5.4 Cardinality explosion --- why per-user labels destroy you

This is *the* most important operational fact about metrics, and a guaranteed senior question. **Every unique combination of label values creates a separate time-series**, and each time-series consumes memory and storage.

Suppose `http_requests_total` has labels:

- `method` (5 values: GET/POST/PUT/DELETE/PATCH)
- `endpoint` (20 routes)
- `status` (~15 codes)
- `instance` (10 pods)

Series count = $5 \times 20 \times 15 \times 10$ = **15,000 time-series**. Totally fine.
Now a well-meaning engineer adds `user_id` (1,000,000 users):
Series count = $15{,}000 \times 1{,}000{,}000$ = **15 billion time-series**.
Each series costs roughly a few KB of RAM in the head block plus ongoing disk. Multiply by billions → Prometheus OOMs, queries time out, the cluster falls over. This is **cardinality explosion**, and it is the single most common way teams take down their own monitoring.

```
Series = ∏ (cardinality of each label)

  method(5) × endpoint(20) × status(15) × instance(10)        = 15,000      ✓
  ...       × user_id(1,000,000)                               = 15,000,000,000  ✗
```

Rules of thumb:

- **Never** put unbounded/high-cardinality values (`user_id`, `request_id`, `email`, `session_id`, raw URLs with IDs, full error messages) in metric labels.
- Bound the cardinality: bucket the URL (`/users/:id` not `/users/12345`), cap status to families, keep instance count sane.
- If you *need* per-user investigation, that's a job for **traces/structured logs** (high-cardinality stores), **not metrics**.

Interview takeaway: ”I'd add a user_id label to the metric” is a trap answer that gets you cut. Metrics are for low-cardinality aggregates; high-cardinality per-entity questions belong in traces or wide structured events. Mentioning cardinality explosion unprompted signals real production experience.

5.5 Exemplars --- the bridge from metrics to traces

A histogram bucket tells you ”13 requests took 2–2.5s” but not *which* requests. An **exemplar** attaches a sample trace id (and its value) to a bucket, so a single slow observation carries a pointer to the trace that produced it.

```python
http_request_duration_seconds_bucket{le="2.5"} 13 # {trace_id="abc123"} 2.31
```

In Grafana, you click the spike on the p99 latency graph and it offers ”view exemplar trace” → you jump straight to a Jaeger waterfall of an *actual* slow request. This collapses the metrics→traces pivot from minutes of guessing to one click. Exemplars are the glue that makes the ”metrics tell you *that*, traces tell you *where*” workflow seamless.

6 Logs

A **log** is a timestamped, discrete record of an event. Logs carry the most **detail** of any pillar (”payment failed: insufficient funds, balance=$12.40, requested = 99.99$”) but are the most **expensive** (volume × retention = cost) and the hardest to aggregate.

6.1 Structured logging --- JSON, not free text

The single biggest upgrade to logging is going from human-readable strings to **structured (JSON) logs** with explicit fields. Free text like `2026-06-15 ERROR payment failed for user 789 amount 99.99` requires fragile regex to query. Structured logs are machine-queryable.

```python
# BAD — unstructured, requires regex archaeology to query
logging.error(f"payment failed for user {user_id} amount {amount}")

# GOOD — structured JSON, every field is independently queryable
import structlog
log = structlog.get_logger()

log.error(
    "payment_failed",
    reason="insufficient_funds",
    user_id=user_id,
    amount=amount,
    currency="USD",
    trace_id=current_trace_id(),   # ← links this log to the trace
    span_id=current_span_id(),
)
```

Emitted:

```json
{
  "timestamp": "2026-06-15T10:30:00.123Z",
  "level": "error",
  "event": "payment_failed",
  "service": "payment-service",
  "reason": "insufficient_funds",
  "user_id": "u-789",
  "amount": 99.99,
  "currency": "USD",
  "trace_id": "abc123",
  "span_id": "def456"
}
```

Now you can run `level:error AND reason:insufficient_funds AND amount > 50` in Kibana/Loki, group by `reason`, and join to the trace via `trace_id`. The `event` field is a **stable identifier** (not a templated string with the amount baked in), so it groups cleanly.

6.2 Log levels --- and when to actually use each

Level	When to use	Pages someone?
TRACE/DEBUG	Fine-grained dev diagnostics; verbose. Usually disabled in prod (or sampled).	Never
INFO	Normal significant events: ”order placed,” ”user logged in.” Audit-worthy facts.	Never
WARN	Something unexpected but handled/recovered: retry succeeded, fell back to cache, deprecated API used. Worth watching, not yet broken.	Maybe (trend)
ERROR	A request/operation failed; needs attention but the service is up.	Possibly (rate-based)
FATAL/CRITICAL	The process cannot continue / is crashing.	Yes

Common mistakes: logging at ERROR for things you handle gracefully (creates noise and false alarms), or at INFO for things that should page (so nobody sees them). Calibrate so that a spike in ERROR logs genuinely correlates with user pain. Don't log-and-rethrow the same exception at three layers — you get one error logged three times.

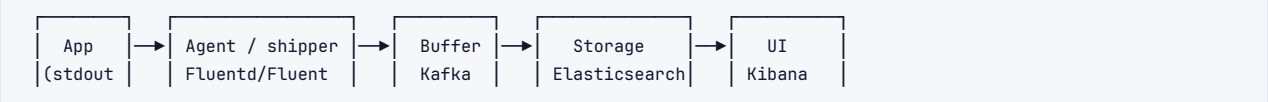
6.3 Sampling logs

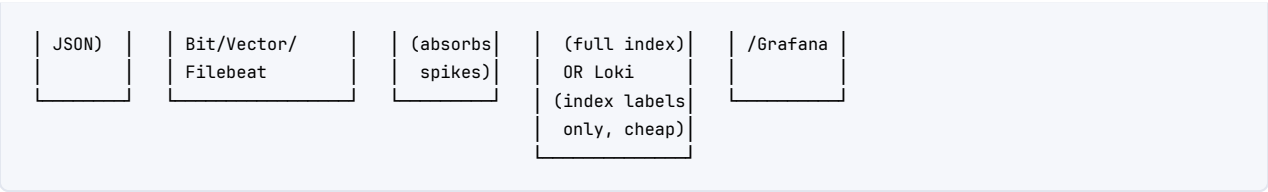
At high volume, logging *every* request is unaffordable (and overwhelms the pipeline). **Sample**:

- **Keep all errors**, sample successes (e.g., 1% of 2xx, 100% of 5xx). Errors are rare and valuable; successes are abundant and cheap to lose.
- **Dynamic sampling**: log more during incidents, less when healthy.
- **Per-key sampling**: keep at least N logs per (endpoint, status) so rare combinations aren't entirely dropped.

6.4 Log aggregation pipeline (ELK / Loki)

You don't grep individual hosts. Logs flow through a pipeline into a central, indexed store.





- **ELK (Elasticsearch + Logstash + Kibana):** full-text inverted index on every field. Powerful arbitrary search, but **storage-heavy and expensive** — you pay to index everything.
- **Loki (Grafana):** indexes only a small set of **labels** (service, level, host) and stores the log body compressed and unindexed. Much cheaper; you grep within a label-narrowed stream rather than full-text-searching everything. Trade-off: less powerful arbitrary search, far lower cost.
- **Kafka as a buffer** decouples producers from the store and absorbs traffic spikes so a log flood doesn't drop data or take down Elasticsearch.

6.5 Correlation IDs --- joining logs ↔ traces ↔ metrics

The thread that stitches the pillars together is a **trace id** (a.k.a. correlation/request id) generated at the edge and propagated everywhere. Put it in **every log line** and **every span**. Then:

1. Alert fires (metric) → grab an exemplar `trace_id`.
2. Open the trace (`trace_id=abc123`) → see the slow span is `payment-service`.
3. Query logs `trace_id:abc123 AND service:payment-service` → read the exact error: `insufficient_funds`.

Without this id you're correlating by timestamp and guessing. With it, all three pillars are one click apart.

6.6 Retention & cost

Logs are the most expensive pillar because cost = **volume** × **retention** × **indexing**. Strategies:

- **Tiered retention:** hot (7 days, fast SSD, fully indexed) → warm (30 days, cheaper) → cold (S3/Glacier, 1 year, for compliance, slow to query).
- **Aggressive sampling** of high-volume low-value logs.
- **Drop noisy fields** before indexing.
- Treat verbose DEBUG as **on-demand** (enable per-service during an incident), not always-on.

6.7 What NOT to log --- PII and secrets

Logs are frequently the source of catastrophic data breaches and compliance violations. **Never log:**

- Passwords, API keys, tokens, session cookies, private keys.
- PII beyond what's necessary: full credit-card numbers (PCI violation), SSNs, full email/address if avoidable. Mask/tokenize (`card_last4="4242"`, not the full PAN).
- Full request/response bodies that may contain any of the above.

Mechanisms: redaction filters in the logging library, allow-lists of safe fields, and automated scanners that fail CI if a log statement references a known-sensitive field. GDPR/CCPA also mean logged PII is subject to deletion requests — another reason to minimize it.

Interview takeaway: "Structured JSON logs, sampled (keep all errors), shipped to a central indexed store, carrying a trace id, with PII redacted and tiered retention" is the complete senior answer to 'how do you do logging.' Mentioning PII/secrets redaction unprompted shows production maturity.

7 Traces

A **distributed trace** records the end-to-end journey of a single request as it fans out across services. It is the pillar that makes microservice latency debuggable — answering "**which hop ate the time?**"

7.1 Spans and span context

The unit of a trace is a **span**: a single named operation with a start time, duration, and metadata. A trace is a **tree of spans**.

Each span carries:

- `trace_id` — identifies the whole request (same across all spans in the trace).
- `span_id` — identifies this span.
- `parent_span_id` — the span that caused this one (null for the root). This is what builds the tree.
- `name` — operation (e.g., `GET /checkout`, `SELECT orders`).
- `start_time`, `duration`.
- **Attributes** (key-value tags): `http.method`, `db.statement`, `user.tier`, `region`. High-cardinality is fine here (unlike metric labels) — this is where per-user debugging lives.
- **Events** (timestamped logs within a span): `"cache miss"`, `"retry attempt 2"`, an exception with stack trace.
- **Status:** OK / Error.

7.2 A trace as a waterfall (tree)

The classic visualization is a **waterfall**: indentation = parent/child depth, horizontal position = start time, bar width = duration.

```
trace_id=abc123                                          total = 240ms
│
├─ gateway: POST /checkout        [████████████████████████]  0─────240ms
│  ├─ auth: validate_token        [███]                        2──20ms
│  ├─ orders: create_order             [██████████]            30──130ms
│  │  ├─ db: INSERT orders              [███]                   35──55ms
│  │  └─ db: SELECT inventory                [████████]         60──130ms ← SLOW SPAN (70ms)
│  └─ payment: charge_card                        [████████]    135──225ms
│     └─ http: POST stripe.com/charge              [██████]    150──215ms (65ms, external)
│
Reading it:
 • Width = time spent. The widest bar that ISN'T just waiting on a child is the culprit.
 • "SELECT inventory" (70ms self-time) and the external Stripe call (65ms) dominate.
 • Gaps between a parent's start and its first child = the parent's own (serial) work.
```

Finding the slow span: look for the span with the largest **self-time** (its own duration minus time waiting on children). A parent that's wide only because its child is wide isn't the problem — the child is. Here, `SELECT inventory` (a missing index?) and the Stripe call are the two real costs; `gateway` is wide simply because it waits for them.

7.3 W3C Trace Context propagation --- the actual header

For the trace to span services, each service must **propagate** the trace context to the next via HTTP headers. The W3C standard is the `traceparent` header:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             │  │                                │                │
             │  └ trace-id (16 bytes, 32 hex)    │                └ trace-flags
             │     same for the whole request    │                   (01 = sampled)
             └ version                           └ parent-id / span-id
                                                    (8 bytes, this span)

tracestate: vendor1=value1,vendor2=value2   # vendor-specific extra context
```

The propagation contract: when service A calls service B, A injects `traceparent` into the outgoing request with A's current span as the `parent-id`. B extracts it, creates a child span referencing A's span, and on its own outbound calls injects *its* span id. The `trace-id` never changes; the `parent-id` does at each hop. The `sampled` flag rides along so the whole trace is consistently kept or dropped.

```python
# OpenTelemetry handles inject/extract automatically, but conceptually:
# Service A (caller):
headers = {}
inject(headers)  # writes traceparent with A's span as parent-id
requests.post("http://service-b/work", headers=headers)

# Service B (callee):
ctx = extract(request.headers)  # reads traceparent
with tracer.start_as_current_span("work", context=ctx):  # child of A's span
    ...
```

If any service in the chain *drops* the header (e.g., a proxy that strips it, or an un-instrumented service), the trace **breaks** into two disconnected trees — a common real-world gap.

7.4 Head vs tail sampling

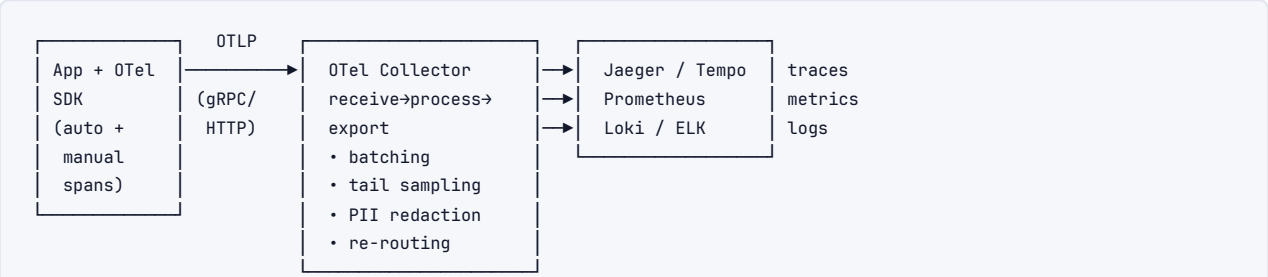
Storing every span at scale is unaffordable, so you **sample**. The decision is *when* you decide to keep a trace.

	Head-based sampling	Tail-based sampling
When	At trace *start* (the root decides, e.g., keep 1%)	At trace *end*, after all spans collected
Cost	Cheap — only sampled traces are ever emitted	Expensive — collector must buffer ALL spans until the trace completes
Catches rare errors?	**No** — a slow/failed trace not chosen at the start is lost forever	**Yes** — can keep 100% of traces that errored or exceeded a latency threshold
Decision propagation	Sampled flag in `traceparent` keeps the whole trace consistent	Collector reassembles, then decides
Typical policy	”keep 1% of everything”	”keep 100% of errors + slow traces + 1% of the rest”

The sweet spot at scale: tail sampling with rules like ”keep every trace that errored or had p99-exceeding latency, plus 0.1% of healthy traces.” This guarantees you capture the *interesting* traces (the whole point of tracing is debugging the bad ones) while keeping cost bounded. The cost is the collector must buffer spans in memory until the root span finishes — non-trivial infrastructure.

7.5 OpenTelemetry --- SDK → Collector → backend

OpenTelemetry (OTel) is the vendor-neutral CNCF standard that unifies instrumentation. You instrument once with the OTel SDK and can export to *any* backend, avoiding vendor lock-in.



- **SDK** — auto-instruments common libraries (HTTP servers, DB drivers, gRPC) so you get spans for free, plus manual spans for business logic.
- **Collector** — a separate process/agent that receives telemetry (OTLP), processes it (batch, sample, drop PII, add resource labels), and exports to backends. Decoupling here means you can switch backends or add tail sampling without touching app code.
- **Backend** — Jaeger/Tempo (traces), Prometheus (metrics), Loki (logs), or a commercial all-in-one (Datadog/Honeycomb).

Python

```python
# Manual span with attributes and an event
from opentelemetry import trace
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("charge_card") as span:
    span.set_attribute("user.tier", "enterprise")   # high-cardinality OK here
    span.set_attribute("amount", 99.99)
    try:
        result = stripe.charge(...)
    except StripeError as e:
        span.add_event("stripe_error", {"code": e.code})  # timestamped event
        span.set_status(trace.Status(trace.StatusCode.ERROR))
        raise
```

Interview takeaway: "Instrument with OpenTelemetry, export via the Collector, tail-sample to keep all errors and slow traces plus a baseline of healthy ones, and propagate W3C traceparent across every hop" is the modern, vendor-neutral, senior-correct answer to 'how do you do tracing.' Naming the traceparent header and tail-vs-head trade-off is a strong differentiator.

8 SLI, SLO, SLA & Error Budgets

This section is where SRE rigor lives, and it's the highest-signal topic in senior interviews. Get the definitions and the math exactly right.

8.1 Precise definitions and how they nest

Term	Definition	Example
SLI (Indicator)	A *measured* number describing service behavior, usually a ratio of good events to total events.	`good = requests < 200ms; SLI = good/total`
SLO (Objective)	The *internal target* you hold yourself to on an SLI, over a window.	”99.9% of requests < 200ms, measured over rolling 28 days”
SLA (Agreement)	The *external, contractual* promise to customers, with **penalties** (refunds/credits) if missed.	”99.9% monthly uptime or 10% bill credit”
Error budget	`1 - SLO` — the *allowed* amount of unreliability. The currency of risk.	`1 - 0.999 = 0.1%` of requests/time may fail

How they nest: `SLI ⊂ SLO ⊂ SLA`.

- The SLI is the raw measurement.
- The SLO is a target *on* that SLI.
- The SLA is set *looser* than the SLO so you have **headroom** — you breach your internal SLO (and react) well before you ever breach the customer contract. If your SLA is 99.9%, your SLO might be 99.95%: by the time you're paying penalties, you've already had alerts firing for a while.

```
Strictest ──────────────────────────────► Loosest
SLO (99.95%)          SLA (99.9%)
"we alert here"       "we owe money here"
internal target       external contract + penalties
     │                      │
     └─ headroom between them = your safety margin
```

8.2 Choosing good SLIs

A good SLI is a **ratio of good events to valid events**, measured *as close to the user as possible* (at the load balancer / edge, not deep in the backend where you miss failures that never reached it). Common categories:

- **Availability:** `successful requests / total valid requests`. (Define ”success” carefully — a 200 with a broken body isn't success; a 404 for a genuinely missing resource isn't *your* failure.)
- **Latency:** `requests faster than threshold / total requests`. (A *threshold-based* ratio, not ”average latency” — it composes into an error budget.)
- **Quality / correctness:** `requests served at full fidelity / total` (e.g., served from primary vs degraded fallback; freshness of data).
- **Freshness/durability** for data systems: `% of data younger than X`, `% of records not lost`.

Don't make an SLI out of something users don't feel (CPU%, internal queue depth). SLIs are about **user-perceived** reliability.

8.3 The nines table --- downtime per year/month/week

`1 - SLO` translated into wall-clock downtime. Memorize the headline rows.

Availability (”nines”)	Error budget	Down / year	Down / 30-day month	Down / week
90% (one nine)	10%	36.5 days	72 hours	16.8 hours
99% (two nines)	1%	3.65 days	7.2 hours	1.68 hours
99.9% (three nines)	0.1%	8.77 hours	**43.2 minutes**	10.1 minutes
99.95%	0.05%	4.38 hours	21.6 minutes	5 minutes

Availability (”nines”)	Error budget	Down / year	Down / 30-day month	Down / week
99.99% (four nines)	0.01%	52.6 minutes	**4.32 minutes**	1 minute
99.999% (five nines)	0.001%	5.26 minutes	**26 seconds**	6 seconds

Error-budget math (the derivation interviewers ask): budget = `(1 - SLO) × window`.

- 99.9% over a 30-day month: `0.001 × 30 days = 0.001 × 43,200 min = 43.2 min` of allowed downtime/badness.
- 99.99% over a month: `0.0001 × 43,200 = 4.32 min`. Notice each extra nine cuts the budget by 10× — and five nines (26s/month) means you can't even do a manual response; it must be fully automated, which is why five nines is extraordinarily expensive.

8.4 Error-budget policy --- what you DO when it's burned

The error budget is only useful if it drives **action**. The **error-budget policy** is the pre-agreed rule:

```
Is there error budget remaining this window?
   ├── YES (within budget) ──➤ ship features, take risks, do experiments,
   │                           run chaos tests, deploy faster
   └── NO  (budget exhausted)➤ FEATURE FREEZE: stop risky deploys,
                               redirect engineering to reliability work,
                               postmortems, hardening — until the
                               window rolls over and budget recovers
```

This is the genius of error budgets: it converts the eternal **dev-vs-SRE argument** (”ship faster!” vs ”stop breaking prod!”) into **arithmetic**. Reliability becomes a shared currency. If you're under budget, even SRE can't block a launch on reliability grounds. If you've blown it, even the PM can't push a risky launch. It aligns incentives instead of pitting teams against each other. The policy must be **agreed and signed off in advance** (by eng + product leadership) so it isn't relitigated mid-incident.

8.5 Multi-window, multi-burn-rate alerting

The hard part is alerting on budget consumption *correctly*. A naive ”alert if any error budget burned” pages constantly. The SRE-canonical solution is **multi-window multi-burn-rate** alerting.

Burn rate = how fast you're consuming the error budget relative to ”even” consumption. Burn rate 1 = you'll exactly exhaust the budget by the end of the window. Burn rate 14.4 = you're burning 14.4× too fast → at this rate the *entire monthly budget* is gone in `30 days / 14.4 ≈ 2 days`.

The trick: pair a **fast (short) window** to react quickly with a **slow (long) window** to confirm it's sustained — so a brief blip doesn't page, but a real sustained burn does, fast.

```
For a 99.9% SLO over 30 days (budget = 0.1%):

┌──────────────┬──────────────┬──────────────┬──────────────────────────┐
│ Severity     │ Burn rate    │ Windows      │ Budget consumed if held  │
│              │ threshold    │ (long+short) │ → action                 │
├──────────────┼──────────────┼──────────────┼──────────────────────────┤
│ FAST burn    │ 14.4×        │ 1h AND 5m    │ ~2% in 1h → PAGE now     │
│ (page)       │              │              │ (budget gone in ~2 days) │
├──────────────┼──────────────┼──────────────┼──────────────────────────┤
│ SLOW burn    │ 1×           │ 6h AND 30m   │ ~10% in 6h → TICKET      │
│ (ticket)     │ (≈1)         │              │ (not urgent, but real)   │
└──────────────┴──────────────┴──────────────┴──────────────────────────┘
```

```
The AND of two windows: the long window proves it's a real sustained burn;
the short window proves it's STILL happening right now (so you don't page on
something that already self-resolved). Both must be over threshold to fire.
```

```promql
# Fast-burn page: 14.4x burn over the last 1h AND still over the last 5m
(
  slo:error_ratio:rate1h  > (14.4 * 0.001)
  and
  slo:error_ratio:rate5m  > (14.4 * 0.001)
)
```

- **Fast burn (14.4×, 1h+5m) → page** someone immediately: at this rate you'll exhaust a month's budget in ~2 days; it's urgent.
- **Slow burn (1×, 6h+30m) → ticket**: a steady low-grade burn that will eventually exhaust the budget but doesn't need a 3am wakeup.

This is dramatically better than threshold alerts because it pages **proportionally to user impact and urgency**, with low false-positive rate, and it ties directly to the SLO rather than to arbitrary resource thresholds.

Interview takeaway: SLI is a measured ratio, SLO is the internal target (stricter than the SLA so you have headroom), SLA is the contract with penalties, error budget is 1−SLO. The budget drives an error-budget *policy* (freeze risky deploys when burned), and you alert with multi-window multi-burn-rate (fast burn → page, slow burn → ticket). Reciting the nines (99.9% ≈ 43 min/month) and the burn-rate concept is a top-tier senior signal.

9 Alerting & On-Call

Alerting converts telemetry into human action. The art is paging **only** when a human must act *now*, and never otherwise.

9.1 Symptom-based vs cause-based alerting

- **Cause-based (bad default):** alert on internal causes — "CPU > 80%," "memory > 90%," "disk queue high." Problem: these are often *fine* (a batch job legitimately maxes CPU; high cache memory is healthy), so they generate **false pages**, and they don't necessarily mean users are hurting.
- **Symptom-based (preferred):** alert on **user-facing symptoms** — "error rate > X," "p99 latency > SLO," "SLO budget burning fast." If users aren't affected, you don't page, *even if a resource looks scary*. This is the core SRE principle.

```
Page on:  "checkout error rate is 5% and SLO budget burning at 14x"   ☑ users hurting
NOT on:   "app-server-7 CPU is at 85%"                                ☐ maybe totally fine
```

Cause-based metrics are still **collected** (you need them to *diagnose* once a symptom alert fires) — they just shouldn't *page* you. Symptoms page; causes inform the investigation.

9.2 Actionable + urgent --- the two-question test

Before any alert is allowed to page, it must pass:

1. **Is it actionable?** Is there something a human can *do right now* in response? If not, it's a dashboard metric, not a page.

2. **Is it urgent?** Does it need attention *this minute*, or can it wait for business hours? If it can wait → ticket, not page.

Only **actionable AND urgent** → page (PagerDuty/Opsgenie). Actionable but not urgent → **ticket**. Neither → **dashboard/log**. This triage is how you kill alert fatigue.

9.3 Reducing alert fatigue

Alert fatigue is when so many alerts fire (especially false ones) that on-call engineers start ignoring or auto-acking them — so the *real* alert gets missed (the "smoke alarm that beeps for toast" → people remove the battery). It's a top cause of bad incidents. Countermeasures:

- Page only on symptoms (above).
- Multi-window burn-rate alerts to suppress transient blips.
- **Deduplication & grouping** (Alertmanager): one root cause that trips 50 services → one page, not 50.
- **Silencing/inhibition:** if the cluster is down, silence the 200 downstream alerts it causes.
- Regularly **review and delete** noisy alerts. An alert that fired 40 times last month and was actioned 0 times must be deleted or fixed.

9.4 Paging vs ticketing

	Page	Ticket
Urgency	Now (wakes a human)	Soon (next business day)
Channel	PagerDuty/Opsgenie → phone	Jira/issue queue
Trigger	Fast-burn SLO, hard outage	Slow-burn SLO, capacity warning, cert expiring in 30 days
Cost of false positive	Very high (burnout, ignored alerts)	Low

9.5 Runbooks

Every page must link to a **runbook**: "what this alert means, how to confirm it's real, the first diagnostic steps, known mitigations, and who to escalate to." A 3am page with no runbook means the on-call rediscovers the system from scratch under stress. Mature teams keep runbooks next to the alert definition.

9.6 Alert as code

Alerts and SLOs are **version-controlled config**, reviewed in PRs, not clicked into a UI. This gives auditability, review, rollback, and reuse.

YAML

```yaml
# Prometheus alerting rule (as code, in git)
groups:
- name: checkout-slo
  rules:
  - alert: CheckoutFastBurn
    expr: |
      slo:checkout:error_ratio:rate1h > (14.4 * 0.001)
      and
      slo:checkout:error_ratio:rate5m > (14.4 * 0.001)
    for: 2m
    labels:
      severity: page
```

```
    annotations:
      summary: "Checkout burning error budget 14.4x (fast)"
      runbook: "https://runbooks.internal/checkout-fast-burn"
```

Interview takeaway: Page only on actionable + urgent symptoms (SLO burn / user-facing errors), route everything else to tickets, attach a runbook to every alert, and manage alerts as code. "Alert on symptoms, not causes" and "every page must be actionable and urgent" are the two phrases that signal you've actually been on-call.

10 Incident Response

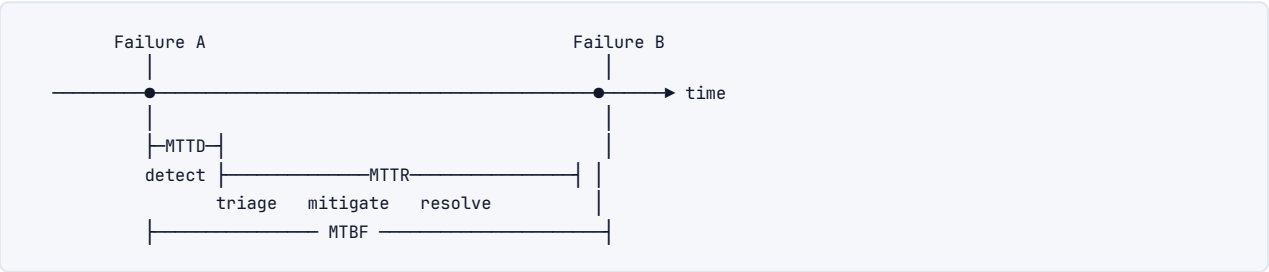
When an alert fires and something's broken, observability feeds a disciplined **incident response** process.

10.1 Severity levels

Most orgs use SEV1–SEV4 (lower number = worse):

Sev	Meaning	Example	Response
SEV1	Critical: major outage, revenue/data loss, most users affected	Checkout fully down; data corruption	All-hands, incident commander, exec comms, war room
SEV2	Major: significant degradation, key feature broken	One region down; p99 5× over SLO	On-call + secondary paged, active mitigation
SEV3	Minor: limited impact, workaround exists	Non-critical feature degraded	Handle in business hours
SEV4	Cosmetic / low	Typo in UI, minor metric gap	Backlog ticket

10.2 The reliability metrics --- MTTD, MTTR, MTBF



- **MTTD (Mean Time To Detect):** failure happens → you *notice* (alert fires). Good observability shrinks this toward zero.
- **MTTR (Mean Time To Recover/Repair):** detect → service restored. The number observability most directly improves — fast triage via metrics→traces→logs.
- **MTBF (Mean Time Between Failures):** how often failures occur. Improved by reliability work (the error-budget freeze redirects effort here).

The goal: **low MTTD + low MTTR + high MTBF.** Note you can have high reliability either by rarely failing (high MTBF) or by recovering instantly (low MTTR) — both count, and at scale fast recovery is often more achievable than never failing.

10.3 The incident lifecycle

1. **Detect** — alert fires (or a human reports it). MTTD clock stops.
2. **Triage** — assess severity, declare the incident, assign an **Incident Commander (IC)** who coordinates (and is *not* the one typing fixes — they run the response). Open a comms channel.
3. **Mitigate — stop the bleeding FIRST.** *Mitigation ≠ root cause.* Roll back the bad deploy, fail over to another region, shed load, flip a feature flag. Restore the user experience *before* you fully understand why. This is the most-missed point: don't debug a SEV1 to root cause while users suffer — mitigate, then investigate.
4. **Resolve** — service fully restored; incident closed. MTTR clock stops.
5. **Postmortem** — write up what happened and prevent recurrence.

10.4 Blameless postmortems

A **blameless postmortem** focuses on *systemic and process* causes, never on punishing individuals. The premise: people act reasonably given the information and tools they had; if a single person's mistake could take down prod, the *system* is at fault (missing guardrail, missing test, missing canary, confusing UI). Blame culture makes people **hide** incidents → you lose the learning → reliability degrades. A good postmortem documents: timeline, impact, root cause(s), what went well, what went poorly, and **action items with owners** (the most important part — a postmortem with no follow-through is theater). The Five Whys is a common root-cause technique. Track whether action items actually get done.

10.5 On-call rotations

- **Rotation:** engineers take turns being primary on-call (e.g., one week each), with a **secondary** as backup/escalation.
- **Follow-the-sun:** distribute on-call across geographies so nobody is paged at 3am — APAC covers their day, then EMEA, then Americas.
- **Sustainability:** the SRE book recommends capping pages per shift (e.g., $\leq$2 actionable incidents per 12h shift) — beyond that, the team is too stretched and must invest in reliability/automation. **Toil budget**: cap operational toil at ~50% so engineers have time to *automate it away*.
- **Compensation & humane practices:** on-call should be compensated, handoffs documented, and a quiet shift should be the norm — a constantly-paging rotation is a bug in the system, not a fact of life.

Interview takeaway: Severity drives response intensity; the lifecycle is detect→triage→mitigate→r*mitigate before you root-cause* (rollback/failover stops user pain fastest, minimizing MTTR); and postmortems are blameless with owned action items. Saying "mitigate first, root-cause later" and "blameless postmortem" shows real incident experience.

11 Debugging with Telemetry (Worked Scenarios)

This is the interview's favorite debugging round. The meta-pattern is always **metrics (that/where in aggregate) → traces (which hop) → logs (why)**, narrowing by dimension at each step. Below are three fully worked walkthroughs.

11.1 Scenario 1 --- "p99 latency spiked 3× at 14:23"

Symptom: Dashboard shows `/checkout` p99 jumped from 200ms to 650ms starting 14:23 UTC. Error rate is *flat* (requests succeed, just slow).

Step 1 — Scope it with metrics (slice by dimension). Before touching traces, narrow *who/what* is affected to avoid boiling the ocean:

```promql
# Is it all endpoints or just one? Slice the p99 by endpoint.
histogram_quantile(0.99,
  sum by (le, endpoint) (rate(http_request_duration_seconds_bucket[5m])))

# Is it all regions / all instances / one bad pod?
histogram_quantile(0.99,
  sum by (le, region) (rate(http_request_duration_seconds_bucket[5m])))
```

Finding: only `/checkout`, all regions, all pods. So it's **endpoint-specific**, not a host or region problem. Cross-check: did `rate()` (traffic) change at 14:23? No traffic spike → not a load problem. Did a **deploy** happen at 14:23? Check the deploy annotations on the dashboard — *yes, checkout-service v412 deployed at 14:22*. Strong suspect.

Step 2 — Find the slow hop with a trace. Click the p99 spike's **exemplar** to jump to an actual slow `/checkout` trace (or query the trace store for `service=checkout duration>600ms`). The waterfall:

```
gateway: POST /checkout          [████████████████████] 640ms
├─ auth                          [■]                    15ms
├─ orders.create                 [████████████████]     520ms ← the new cost
│  └─ db: SELECT inventory         [██████████████]      480ms ← self-time here
└─ payment                       [■■]                   80ms
```

The `SELECT inventory` span now takes 480ms (was ~40ms). The latency lives in **one database query** in `orders.create`.

Step 3 — Find *why* with logs / the span attributes. Read the span's `db.statement` attribute or query logs `trace_id:<that trace>`:

```json
{"event":"slow_query","trace_id":"...","db.statement":"SELECT * FROM inventory WHERE sku = ? AND
  ↪  warehouse_id = ?","rows_scanned":2400000,"index_used":null}
```

`index_used: null`, `rows_scanned: 2.4M` → a **full table scan**. The v412 deploy added a `warehouse_id` filter to the inventory query, but there's no composite index on `(sku, warehouse_id)` → the query degraded from index seek to full scan.

Root cause: new query path missing an index. **Mitigation:** roll back v412 (fastest — restores p99 immediately). **Fix:** add the composite index, then re-deploy. **Verify:** p99 back to 200ms; the slow-query log gone.

> The discipline: *narrow by dimension in metrics first* (endpoint? region? deploy?), *then* one trace pinpoints the hop, *then* logs/attributes give the why. You did not guess once.

11.2 Scenario 2 --- "Error rate climbing" (drill down by dimension)

Symptom: Overall 5xx rate climbing from 0.1% to 4% over 20 minutes. SLO fast-burn alert paged.

Step 1 — Decompose the error rate by every dimension to find the concentrated slice:

```promql
# By status code — what KIND of error?
sum by (status) (rate(http_requests_total{status=~"5.."}[5m]))
#   → 503s dominate (not 500s) → upstream unavailable, not a code bug

# By endpoint — is it everywhere or one route?
sum by (endpoint) (rate(http_requests_total{status=~"5.."}[5m]))
#   → concentrated on endpoints that call payment-service

# By downstream dependency / region / version
sum by (downstream) (rate(http_requests_total{status="503"}[5m]))
#   → all 503s are from calls to payment-service
```

The errors are **503s, on payment-dependent endpoints, all pointing at payment-service.** The blast radius is now precise: payment-service is the culprit.

Step 2 — Trace a failing request (`status=503`, tail-sampled so errors are kept):

```
gateway: POST /checkout            [■■■■■■■]  status=ERROR
├─ orders                          [■■]       OK
└─ payment.charge                  [■■■■]     status=ERROR  ← fails here
   └─ http POST payment-svc        [×]        connection refused / timeout
```

The error originates at the call into payment-service: connection refused / timeouts.

Step 3 — Logs + RED on payment-service itself. Pivot to payment-service's own dashboards and logs:

- payment-service `rate()` of requests is normal, but its **own** error logs show `OutOfMemoryError` / pod restarts; `kube-state-metrics` shows `container_restarts` climbing for payment-service at the same timestamp.
- Logs `service:payment-service level:error` → `OOMKilled` events. A memory leak (or a too-low memory limit after a config change) → pods OOM, restart, drop connections → upstream sees 503s.

Root cause: payment-service pods OOMKilled in a crash-loop. **Mitigation:** bump the memory limit / roll back the change that raised memory use / scale out to spread load while restarting. **Then** fix the leak. **Defense:** a **circuit breaker** in the gateway would have failed fast and shed load instead of piling connections onto the dying service — note this in the postmortem.

> Pattern: when error rate climbs, **decompose by dimension** (status × endpoint × downstream × region × version) until the errors *concentrate* in one slice — that slice *is* the lead. Then trace one failure to confirm the hop, then logs/restart-metrics for the why.

11.3 Scenario 3 --- Profiling a CPU hotspot (continuous profiling, flame graphs, eBPF)

Symptom: A service's CPU is at 90% and p99 is creeping up, but traces show no slow *downstream* span — the time is spent *inside the service's own code* (high self-time on the root span, no

child explains it). Traces tell you *which service*; they don't tell you *which function*. For that you need **profiling**.

Continuous profiling runs a low-overhead sampling profiler *in production all the time* (Parca, Pyroscope, Google-Wide Profiling, `pprof`). It samples each thread's call stack ~100×/sec; the aggregate over thousands of samples shows where CPU time actually goes — as a **flame graph**.

```
Flame graph (CPU; wider = more CPU time; bottom = root)

main ──────────────────────────────────────────────────────────
serve_request ─────────────────────────────────────────
parse_request ─── render_response ──────────────  log_event ──
              json.Marshal ──────────────────────
                            reflect.Value.* ████████████  ← HOT
                            (reflection in JSON encoding = 60% of CPU)
```

The widest plateau at the top is the hottest code. Here `reflect.*` under `json.Marshal` is eating 60% of CPU — the service is serializing a huge response via reflection-based JSON on every request. Fix: switch to a codegen'd/streaming serializer, or cache the serialized payload. CPU drops, p99 recovers.

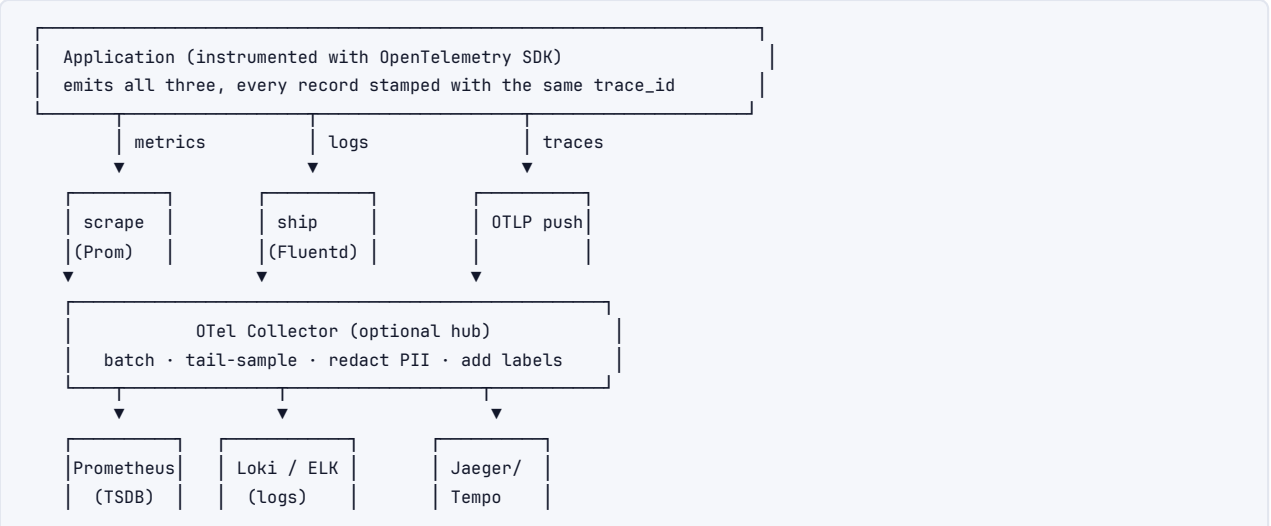
Reading flame graphs: widest bars at the top = hottest; a flat plateau means *that* function itself is the bottleneck (time not attributed to its callees). Profiles **differential** beautifully — diff "before deploy" vs "after deploy" flame graphs to see exactly which function got more expensive.

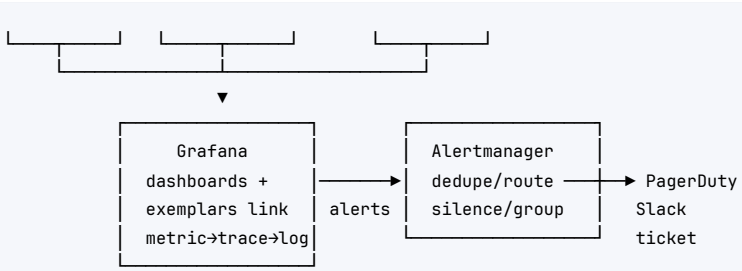
eBPF (brief): eBPF lets you run sandboxed programs **in the Linux kernel** to capture system-wide telemetry with near-zero overhead and **zero application instrumentation** — CPU profiles, off-CPU time (where threads *block*, e.g., on locks or I/O — invisible to a CPU profiler), syscall latency, network flows. Tools: `bcc`, `bpftrace`, Pixie, Parca-agent, Cilium (eBPF networking). It's how you profile and trace *without* touching the app — increasingly the foundation of modern observability agents.

Interview takeaway: When the trace shows the time is *inside* a service (high self-time, no slow child), reach for a continuous profiler and read the flame graph — widest top bar is the hotspot. eBPF gives whole-system, zero-instrumentation profiling (including off-CPU/blocking time). This is the fourth pillar ("profiles") and naming it differentiates you.

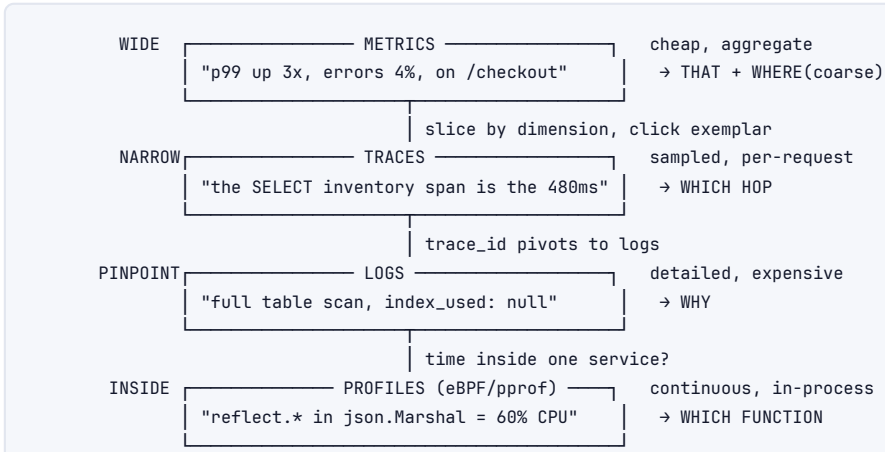
12 Architecture / Diagrams

12.1 The unified telemetry pipeline (three pillars, one trace id)

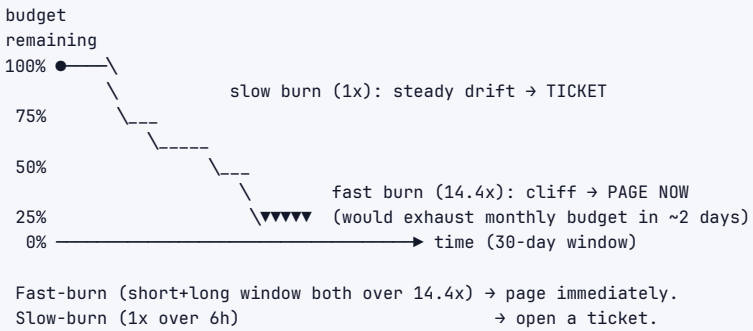




12.2 The investigation funnel (how the pillars compose)



12.3 Error-budget burn (how multi-burn-rate alerts fire)



13 Real-World Examples

Tool / System	Pillar	Notes
Prometheus + Grafana	metrics + dashboards	The de-facto open-source standard. Pull model, PromQL, TSDB.
Thanos / Cortex / Mimir	metrics (long-term, HA)	Make Prometheus globally queryable and durable at scale.
ELK (Elasticsearch/Logstash/Kibana)	logs	Full-text index; powerful but storage-heavy.
Grafana Loki	logs	Indexes labels only; cheap; ”Prometheus for logs.”

Tool / System	Pillar	Notes
Jaeger / Zipkin / Grafana Tempo	traces	Distributed tracing backends; Tempo is cheap object-storage-backed.
OpenTelemetry	all three	Vendor-neutral instrumentation standard + Collector. The CNCF default.
Datadog / New Relic / Dynatrace	all-in-one (commercial)	Single pane of glass; pay for convenience and scale.
Honeycomb	events / high-cardinality	Popularized "observability"; `BubbleUp` slices by arbitrary dimensions.
Pyroscope / Parca / Polar Signals	profiles	Continuous profiling, flame graphs, eBPF-based.
PagerDuty / Opsgenie	alerting/on-call	Routing, escalation, schedules.
Google SRE	the model	Origin of SLI/SLO/error budget, blameless postmortems, toil.
AWS CloudWatch / X-Ray, GCP Cloud Monitoring/Trace	cloud-native	Managed equivalents on the big clouds.

14 Real-Life Analogies

One theme — running a hospital ER — every observability concept maps to a piece of patient care.

Concept	Analogy
Metrics	The bedside vitals monitor: heart rate, blood pressure, O_2 — continuous numbers that tell you *something is wrong* at a glance, but not why.
Logs	The patient's detailed chart: every event, timestamped — "10:03 administered 5mg, 10:07 reported nausea." Full detail, but you read it after the monitor alarms.
Traces	Following one patient's entire journey through the hospital — ER → X-ray → lab → surgery → recovery — and timing each stop to find where they got stuck waiting.
Profiling	A surgeon's intra-operative imaging: zooming *inside* one organ (one service) to see exactly which tissue (function) is the problem.
Monitoring vs observability	Monitoring = the alarms you pre-set for known dangers (low O_2). Observability = a senior doctor who can diagnose a *novel* illness from the same instruments by asking new questions.
Cardinality explosion	Trying to keep a separate live monitor for every individual patient who ever visited — the room fills with screens until the whole system collapses. Keep aggregate vitals; pull the individual chart only when needed.
SLO / error budget	The ER's target: "95% of patients seen within 30 min." The 5% slack is your budget — spend it on training drills when you're under it; when you've blown it, all hands stop side-projects and focus on throughput.
SLA	The contract with the insurer: miss the target and you pay penalties — set looser than your internal target so you react before it costs money.
Burn-rate alerting	If patients are backing up 14× faster than normal, page the on-call attending *now*; a slow steady backlog just goes on tomorrow's planning list.
Alert fatigue	A monitor that beeps for every tiny movement — nurses stop reacting, so the real cardiac arrest gets missed.
Symptom vs cause alerting	Page when the *patient* is in distress (symptom), not merely because a machine's fan is loud (cause that may be harmless).
Mitigate before root-cause	Stabilize the patient (stop the bleeding) *first*; figure out the underlying disease *after* they're out of danger.

Concept	Analogy
Blameless postmortem	The M&M (morbidity & mortality) conference: examine what the *system* allowed, not which nurse to blame — so people report mistakes instead of hiding them.
Trace id linking pillars	The patient's wristband ID — scan it and instantly pull their vitals, chart, and journey together.
Tail sampling	You can't film every patient's whole visit, but you *do* keep full footage of everyone who coded or waited too long — the interesting cases.

15 Memory Tricks / Mnemonics

- **Three pillars — "MLT"** (like a sandwich): **M**etrics (what), **L**ogs (detail), **T**races (where). Add **P**rofiles for inside-a-service → "MLTP."
- **The investigation order — "Metrics say THAT, Traces say WHERE, Logs say WHY, Profiles say WHICH FUNCTION."**
- **RED** (services): **R**ate, **E**rrors, **D**uration → "RED light = stop and check the service."
- **USE** (resources): **U**tilization, **S**aturation, **E**rrors → "Uncle Sam's Equipment."
- **Four Golden Signals — "LTES"**: Latency, Traffic, Errors, Saturation (= RED ∪ USE).
- **SLI ⊂ SLO ⊂ SLA** — **I**ndicator (measured), **O**bjective (internal target), **A**greement (external contract). "Indicator, Objective, Agreement — small to big."
- **Error budget = 1 − SLO.** "Spend it on speed; freeze when it's gone."
- **The nines — "three-nines is 43 minutes a month."** Each extra nine ÷10: 99.9% → 43 min, 99.99% → 4.3 min, 99.999% → 26 s.
- **Alerting — "actionable AND urgent → page; else ticket; else dashboard."**
- **Alert on symptoms, not causes** — "page on the patient, not the machine."
- **Cardinality — "never label a metric with anything unbounded"** (user_id, request_id, email).
- **Histograms aggregate, summaries don't** — "you can't average percentiles."
- **Incident lifecycle — "DTMRP": Detect → Triage → Mitigate → Resolve → Postmortem; mitigate before root-cause.**
- **traceparent header — "version-trace-parent-flags"** (`00-<traceid>-<spanid>-01`).

16 Common Interview Questions

16.1 Q1: What are the three pillars of observability, and when do you use each?

Model answer: **Metrics** are cheap, pre-aggregated time-series (rate/errors/latency via counters, gauges, histograms) — ideal for dashboards, SLOs, and alerting; their weakness is cardinality, so they answer "what/how much" in aggregate. **Logs** are detailed, timestamped, discrete events for "what exactly happened" — prefer structured JSON so they're queryable; they're the most expensive pillar so you sample (keep all errors). **Traces** record one request's path across services as a tree of spans for "where did the time/error occur" — essential in microservices. The workflow ties them together: a metric alert tells you *something's wrong*, a trace shows you *which hop*, logs tell you *why* — all linked by a shared trace id.

Follow-ups:

- *Add a fourth pillar?* → Continuous **profiles** (flame graphs / eBPF) for "which function" when the time is inside a single service.
- *Which is most expensive and why?* → Logs (volume × retention × indexing); control with sampling + tiered retention.

16.2 Q2: Monitoring vs observability --- what's the difference?

Model answer: **Monitoring** watches **known** failure modes via pre-defined metrics/dashboards/alerts — it answers "is the thing I anticipated broken?" (known-knowns and known-unknowns). **Observability** is the *property* of a system that lets you ask **new** questions and debug **unknown-unknowns** from the telemetry you already emit, *without shipping new code* — "why is p99 up only for EU Android users on /checkout paying with PayPal?" Monitoring is a subset of observability; observability requires richer, high-cardinality, correlatable data (the pillars linked by a trace id).

Follow-ups:

- *Why can't classic metrics give observability?* → Cardinality — you can't store a series per user; high-cardinality slicing needs traces/structured events.
- *Give a concrete unknown-unknown.* → A bug only triggered by a specific feature-flag + device + region combination nobody dashboarded.

16.3 Q3: Counter vs gauge vs histogram vs summary --- and how do you get p99 from a histogram?

Model answer: A **counter** only increases (requests_total) — you read its `rate()`. A **gauge** goes up and down (queue depth, memory). A **histogram** buckets observations into cumulative `le` buckets so you compute percentiles server-side with `histogram_quantile()`, which finds the bucket containing the target rank and linearly interpolates within it. A **summary** precomputes quantiles client-side. The crucial difference: **histogram buckets aggregate across instances** (sum the buckets, then compute the quantile), whereas **summary quantiles cannot be merged** — you can't average percentiles — so for multi-instance services you must use histograms. Accuracy depends on bucket placement, so put buckets near your SLO threshold.

Follow-ups:

- *Your p99 looks wrong — why?* → Wide bucket around the percentile → coarse interpolation; add buckets near that latency.
- *Cost of many buckets?* → Each bucket is a series → cardinality; balance precision vs cost.

16.4 Q4: Explain SLI, SLO, SLA, and error budget. How do they relate?

Model answer: An **SLI** is a measured indicator, usually good-events/valid-events (e.g., % of requests < 200ms). An **SLO** is the internal target on that SLI (99.9% over 28 days). An **SLA** is the external, contractual promise with penalties (refunds), set *looser* than the SLO so you have headroom and react before you owe money. The **error budget** is `1 - SLO` — the allowed unreliability (0.1% ≈ 43 min/month). They nest SLI ⊂ SLO ⊂ SLA. The budget is a currency: within it you ship fast and take risks; once burned, an **error-budget policy** freezes risky deploys and redirects effort to reliability — turning the dev-vs-SRE tension into arithmetic.

Follow-ups:

- *Why is the SLO stricter than the SLA?* → Headroom: alert and fix before breaching the contract.
- *Who sets the error-budget policy?* → Eng + product leadership, in advance, so it isn't relitigated mid-incident.

16.5 Q5: How do you alert on an SLO without paging constantly?

Model answer: Use **multi-window, multi-burn-rate** alerting. Burn rate is how fast you're consuming the budget relative to even consumption; burn rate 1 exactly exhausts the budget over the window. Pair a short and a long window with an `AND`: the long window confirms it's a sustained real burn, the short window confirms it's *still happening now*. A **fast burn** (e.g., 14.4× over 1h AND 5m → ~2% of monthly budget in an hour) **pages** immediately; a **slow burn** (1× over 6h AND 30m) opens a **ticket**. This pages proportionally to user impact and urgency with

low false positives, tied to the SLO rather than arbitrary resource thresholds.

Follow-ups:

- *Why two windows?* → Long = real/sustained; short = currently active (don't page on something already resolved).
- *Where does 14.4 come from?* → It's the burn rate that consumes ~2% of a 30-day budget in 1 hour (a common page threshold from the SRE workbook).

16.6 Q6: Walk me through debugging a 3× p99 latency spike.

Model answer: First, **scope with metrics**: slice p99 by endpoint, region, instance, and version to find the affected slice; check if traffic spiked (load problem) and whether a deploy coincided (correlate with deploy annotations). Suppose it's one endpoint, all hosts, right after a deploy. Second, **find the slow hop with a trace** — click the latency exemplar or query traces by `duration > threshold`; the waterfall shows which span has the high *self-time* (say, a DB query). Third, **find why with logs / span attributes** — the `db.statement` and `rows_scanned`/`index_used` reveal a full table scan from a new query missing an index. **Mitigate by rolling back** (fastest), then add the index and re-deploy, then verify p99 recovered. The discipline: narrow by dimension first, one trace pinpoints the hop, logs give the cause — never guess.

Follow-ups:

- *Error rate is flat but latency is up — what does that tell you?* → Requests succeed slowly → not a crash; suspect a slow dependency/query/GC, not a logic bug.
- *No deploy happened — now what?* → Look at traffic (load/saturation), downstream latencies, GC pauses, noisy-neighbor, or data growth (a query that scales with table size).

16.7 Q7: How does distributed tracing propagate context across services?

Model answer: At the edge, a **trace id** and root **span id** are generated. Each service, on an outbound call, **injects** the W3C `traceparent` header — `00-<32-hex trace-id>-<16-hex parent-span-id>-<flags>` — where the parent-span-id is its current span and the flags carry the sampled bit. The callee **extracts** it, creates a child span referencing the parent, and on *its* outbound calls injects its own span id. The trace-id is constant across the whole request; the parent-span-id changes per hop; the sampled flag keeps the whole trace consistently kept or dropped. The backend reassembles spans by trace-id/parent-id into a waterfall. OpenTelemetry's SDKs handle inject/extract automatically; if any hop strips the header, the trace breaks into disconnected trees.

Follow-ups:

- *Head vs tail sampling?* → Head decides at start (cheap, misses rare errors); tail decides at end after buffering all spans (expensive, keeps all errors/slow traces). At scale: tail-sample errors + slow + a baseline.
- *How do logs join the trace?* → Stamp every log with the trace id; pivot `trace_id:abc` across both stores.

16.8 Q8: Why percentiles, not averages, for latency?

Model answer: Averages hide the tail — a handful of very slow requests barely move the mean but dominate the worst user experiences. **p95/p99/p99.9** show what your slowest users actually feel, and at scale a "1%" tail is millions of requests. Worse, in fan-out architectures the tail *amplifies*: a request hitting 100 backends will likely hit at least one slow one ($1 - 0.99^{100} \approx 63\%$), so a backend's p99 becomes the system's p50. You therefore define SLOs and alerts on percentiles and design tail-tolerant patterns (hedged requests, timeouts). Also: never *average percentiles* across instances — aggregate the histogram buckets first.

Follow-ups:

- *Which percentile for an SLO?* → Usually p99 (sometimes p99.9) for user-facing; depends on how much tail users feel.
- *Average can even be misleading downward — how?* → A flood of fast-failing 500s lowers average latency; split success vs error latency.

16.9 Q9: What should and shouldn't go into a log line?

Model answer: Should: structured JSON with a stable `event` name, a `level`, the `service`, a `trace_id`/`span_id` for correlation, and the specific fields you'd query on (`reason`, `user_id`, `amount`). **Shouldn't:** secrets (passwords, tokens, keys, cookies) and PII beyond necessity (full card numbers → PCI violation, SSNs, raw bodies that may contain them) — redact/mask (`card_last4`). Calibrate **levels** (don't log handled cases at ERROR), **sample** (keep all errors, sample successes), and use **tiered retention** to control cost. Logged PII is also subject to GDPR deletion — another reason to minimize it.

Follow-ups:

- *How do you enforce no-PII?* → Redaction filters in the logging lib, field allow-lists, and CI scanners that fail builds referencing sensitive fields.
- *ELK vs Loki?* → ELK full-text-indexes everything (powerful, costly); Loki indexes labels only and greps the body (cheap, less arbitrary search).

16.10 Q10: Your metric storage is exploding. What happened and how do you fix it?

Model answer: Almost certainly **cardinality explosion** — someone added a high-cardinality label (user_id, request_id, email, raw URL with IDs, full error message). Series count is the *product* of every label's cardinality, so adding a 1M-value label multiplies series by a million and OOMs the TSDB. Fix: drop/relabel the offending label, bound dimensions (template the URL to `/users/:id`, collapse status to families), and move per-entity investigation to **traces/structured events** (high-cardinality stores) rather than metrics. Prevent recurrence with cardinality limits/linting in the metrics pipeline.

Follow-ups:

- *But I need per-user debugging.* → That's a trace/event query (`group by user_id` on raw events), not a metric label.
- *How to detect which label is the culprit?* → Prometheus `topk` on series count by label; `count by (__name__)` and per-label cardinality reports.

16.11 Q11: Describe the incident lifecycle. What's the most common mistake?

Model answer: Detect (alert fires — MTTD) → **Triage** (assess severity, declare the incident, assign an Incident Commander, open comms) → **Mitigate** (stop the bleeding — rollback, failover, shed load, flip a flag) → **Resolve** (full service restored — MTTR) → **Postmortem** (blameless, with owned action items). The most common mistake is **trying to root-cause before mitigating** — you should restore the user experience *first* (rollback/failover) and investigate *after*, because every minute spent debugging a live SEV1 is MTTR users feel. Also: postmortems must be blameless (focus on systemic gaps) and produce tracked action items, or you don't actually improve.

Follow-ups:

- *What does the Incident Commander do?* → Coordinates the response and comms; does *not* type the fix — keeps the response organized.
- *MTTD vs MTTR vs MTBF?* → Time to detect, to recover, between failures; observability shrinks MTTD/MTTR, reliability work raises MTBF.

16.12 Q12: How would you add observability to a system you just designed?

Model answer: I'd define the **SLIs** that matter to users (availability and p99 latency on the critical path, plus quality/freshness if relevant), set an **SLO** with an **error budget** and a freeze **policy**. For the three pillars: **RED metrics** per service (and USE on resources/queues) via Prometheus, **structured JSON logs** with trace ids shipped to Loki/ELK with PII redaction and tiered retention, and **OpenTelemetry traces** propagating `traceparent` across hops, tail-sampled to keep all errors and slow traces. I'd alert **symptom-based** with **multi-window burn-rate** (fast → page, slow → ticket), attach **runbooks**, manage alerts as code, and link the pillars by trace id (exemplars from metrics → traces → logs). Continuous profiling for in-service hotspots.

Follow-ups:

- *Cost concerns?* → Sample logs/traces (keep all errors), bound metric cardinality, tier retention.
- *How do you know your SLIs are right?* → They must correlate with real user pain; validate against support tickets / user-reported issues.

17 Senior-Level Discussion Points

- **Observability is a design-time requirement, not an afterthought.** You can't retrofit a trace id; instrument at the boundaries from day one and treat telemetry schema (metric names, label sets, span attributes) as an API you version.
- **The cardinality/observability tension is the central engineering trade-off.** Debugging unknowns needs high-cardinality dimensions, but metrics can't afford them. The resolution is a *tiered* strategy: low-cardinality metrics for alerting/SLOs, high-cardinality traces/events for exploration, with sampling and cost controls. Knowing *why* you can't just "label everything" is the senior signal.
- **Error budgets are an organizational tool, not just a number.** Their value is resolving the dev-vs-SRE conflict by making reliability a shared, quantified currency. The policy (signed off in advance) is what gives it teeth.
- **Symptom-based alerting + SLO burn rate scales; per-resource threshold alerting does not.** As services multiply, cause-based alerts produce unmanageable noise. Tie pages to user-facing SLO burn.
- **Telemetry has real cost and the bill grows super-linearly with scale.** Logs especially. Senior engineers treat observability cost as a budget to optimize (sampling, retention tiers, dropping unused metrics, Loki-vs-ELK) — at FAANG scale observability can rival the cost of the service it observes.
- **Mitigation decoupled from diagnosis.** Build for fast rollback/failover/feature-flags so MTTR is low *independent* of how long root-causing takes. Resilience patterns (circuit breakers, load shedding, bulkheads) reduce blast radius before observability even gets involved.
- **Tail sampling and exemplars are what make tracing useful at scale** — keep the *interesting* traces (errors, slow) not a blind 1%, and wire exemplars so metrics spikes link straight to a representative trace.
- **eBPF and continuous profiling are the frontier** — whole-system, zero-instrumentation telemetry (including off-CPU/blocking time) is increasingly the foundation of agents (Pixie, Parca, Cilium).
- **Cross-team trace continuity is an org problem.** One un-instrumented or header-stripping service breaks every trace through it; observability requires platform-level standards (OTel everywhere, propagation contracts) that span team boundaries.

18 Typical Mistakes Candidates Make

1. **Finishing a system design with no observability story.** "How do you know it's healthy / how do you debug it?" must be answered proactively. This is the #1 omission that signals junior.
2. **Using averages instead of percentiles** for latency (and not knowing you can't average percentiles across instances).
3. **Alerting on causes (CPU/memory) instead of symptoms (SLO burn / user errors)** → noisy, non-actionable pages and alert fatigue.
4. **Proposing per-user (or per-request) metric labels** — cardinality explosion that takes down the monitoring stack. High-cardinality belongs in traces/events.
5. **Unstructured logs** (free text needing regex) and logging too much (cost/PII) or too little (blind).
6. **No trace id linking the pillars** — can't pivot from a metric to the trace to the logs for one request.
7. **Confusing SLA and SLO** (or setting them equal — no headroom), or having an SLO with no error-budget *policy*.
8. **Root-causing before mitigating** during an incident — debugging a live SEV1 while users suffer instead of rolling back first.
9. **Conflating monitoring and observability** as synonyms — missing the known vs unknown-unknown distinction.
10. **Choosing a summary when a histogram is needed** (multi-instance services) and being unable to compute a fleet-wide percentile.
11. **No runbooks** on alerts, so a 3am page means rediscovering the system under stress.
12. **Ignoring telemetry cost** — proposing "log everything, trace everything at 100%" with no sampling or retention strategy.

19 How This Connects to Other Topics

Topic	Connection
Performance Engineering (§09)	Percentiles, tail latency, fan-out amplification, flame graphs, USE/RED — observability is how you *measure* the performance you optimize. Profiling is the shared tool.
Distributed Systems (§07)	Tracing exists *because* of distributed systems; context propagation, partial failures, and fan-out are the problems observability solves.
Cloud & Infrastructure (§04)	Prometheus/Grafana/OTel/Loki/Jaeger run on your K8s platform; kube-state-metrics, node_exporter, service-mesh tracing (Envoy) all feed telemetry.
System Design (§05)	Every design needs an SLI/SLO + three-pillars + alerting section; capacity planning uses the same metrics.
Reliability / SRE	SLOs, error budgets, blameless postmortems, on-call, toil budgets — observability is SRE's instrument panel.
Networking	Blackbox probing, trace spans for RPC hops, packet-drop USE-errors, TLS-cert-expiry alerts.
Security	Audit logs, anomaly detection, PII/secret redaction in logs, access logs for forensics.
Testing (§17)	Monitoring catches in production what tests miss; canary analysis uses SLIs; synthetic monitoring is testing in prod.

Topic	Connection
Databases	Slow-query logs, query plans (`EXPLAIN`), connection-pool saturation metrics, replication-lag SLIs — observability surfaces DB bottlenecks.

20 FAANG Interview Tips

1. **End every system design with "here's how I'd observe it":** name the key SLIs, set an SLO with an error budget, the three pillars linked by a trace id, and symptom-based alerts. This single move dramatically raises your level signal.
2. **Say "p99, not average"** and **"alert on symptoms / SLO burn rate, not on CPU."** These two phrases instantly read as senior.
3. **Mention an error-budget policy** ("freeze risky deploys when the budget's burned") to show you understand the velocity-vs-reliability trade-off.
4. **Name the cardinality trap unprompted** — "I wouldn't put user_id in a metric label; that's a cardinality explosion — per-user lives in traces." Strong production signal.
5. **Use the metrics→traces→logs (→profiles) funnel** when asked to debug. Narrow by dimension first; never guess.
6. **Know the nines** cold: 99.9% ≈ 43 min/month, each extra nine ÷10.
7. **Mitigate before root-cause** during incident questions — rollback/failover first, investigate after.
8. **Histograms aggregate, summaries don't** — drop this when metric types come up.
9. **Name OpenTelemetry and the traceparent header** for tracing questions; mention head vs tail sampling.
10. **Connect to business impact** — "good observability cuts MTTR, and at Amazon-scale every minute of downtime is real revenue." Tie telemetry to money.

21 Revision Cheat Sheet

21.1 10-Minute Summary

Observability = understand a system's internals from its outputs, so you can debug **unknown** failures without shipping code. **Monitoring** (known failure modes) is a subset. Built on three pillars: **Metrics** (cheap aggregates → *that* something's wrong), **Traces** (one request across services → *where*), **Logs** (detailed events → *why*) — plus **Profiles** (→ *which function*). Link them all by a **trace id**.

Metrics: counter (only up, read `rate()`), gauge (up/down), **histogram** (buckets → `histogram_quantile()`, *aggregatable* across instances), summary (client-side quantiles, *not* aggregatable — never average percentiles). Prometheus **pulls** from `/metrics` (failed scrape = down signal); Pushgateway for short jobs. **RED** per service, **USE** per resource, **Four Golden Signals** = union. **Cardinality explosion**: series = product of label cardinalities — never label with user_id/request_id.

Logs: structured JSON, stable event name, trace id, leveled (calibrated), sampled (keep all errors), shipped to ELK (full index, costly) or Loki (label index, cheap), PII/secrets redacted, tiered retention.

Traces: spans form a tree; find the span with high **self-time**. Propagate **W3C `traceparent`** (`00-<traceid>-<spanid>-<flags>`) across hops. **Head sampling** (cheap, misses errors) vs **tail sampling** (keep all errors/slow, expensive). **OpenTelemetry**: SDK → Collector → backend.

SLI ⊂ SLO ⊂ SLA. SLI = measured ratio; SLO = internal target (stricter than SLA for headroom); SLA = contract + penalties. **Error budget = 1 − SLO** (99.9% ≈ 43 min/month). **Policy**:

freeze risky deploys when burned. **Multi-window multi-burn-rate** alerts: fast burn (14.4×, 1h+5m) → page; slow burn (1×, 6h+30m) → ticket.

Alerting: symptoms not causes; actionable AND urgent → page, else ticket; runbooks; alerts as code; kill fatigue. **Incidents:** detect→triage→mitigate→resolve→postmortem; **mitigate before root-cause**; blameless postmortems with owned actions; MTTD/MTTR/MTBF.

21.2 Key Numbers

Thing	Value
99%	7.2 h/month down
99.9%	**43.2 min/month**
99.95%	21.6 min/month
99.99%	**4.32 min/month**
99.999%	26 s/month
Fast-burn page	14.4× over 1h AND 5m
Slow-burn ticket	1× over 6h AND 30m
Prometheus scrape	default 15s
Fan-out p99	100 backends @ 1% slow → ~63% hit a slow one

21.3 Cheat Sheet Table

Concept	One-liner
Three pillars	Metrics (what/that) · Traces (where) · Logs (why) · +Profiles (which fn)
Monitoring vs observability	known failures vs debugging unknown-unknowns; monitoring ⊂ observability
Metric types	counter↑ · gauge↕ · histogram(buckets, aggregatable) · summary(quantiles, not)
Pull model	Prometheus scrapes /metrics; failed scrape = down; Pushgateway for short jobs
RED / USE / Golden	Rate-Errors-Duration · Util-Sat-Errors · Latency-Traffic-Errors-Saturation
Cardinality	series = ∏ label cardinalities; never user_id in a metric label
Exemplar	metric bucket → pointer to a real trace
Structured logs	JSON + trace_id; keep all errors, sample successes; redact PII
ELK vs Loki	full-text index (costly) vs label index + grep (cheap)
Trace	tree of spans; find high self-time span
traceparent	`00-<traceid>-<spanid>-<flags>`; propagate across hops
Head vs tail sampling	start/cheap/miss-errors vs end/expensive/keep-errors
OpenTelemetry	SDK → Collector → backend; vendor-neutral
SLI ⊂ SLO ⊂ SLA	measured ratio · internal target · external contract+penalties
Error budget	1 − SLO; spend on velocity, freeze when gone
Burn-rate alerts	fast(14.4×,1h+5m)→page · slow(1×,6h+30m)→ticket
Symptom alerting	page on user impact, not CPU; actionable + urgent
Incident	detect→triage→mitigate→resolve→postmortem; mitigate first; blameless
MTTD/MTTR/MTBF	detect · recover · between failures

21.4 Most Important Concepts

1. **Metrics say *that*, traces say *where*, logs say *why*** — linked by a trace id; the universal debugging funnel.
2. **Monitoring = known failures; observability = unknown-unknowns** (monitoring is a subset).
3. **Histograms aggregate, summaries don't** — you can't average percentiles.
4. **Cardinality explosion** — never put unbounded labels on metrics.
5. **SLI $\subset$ SLO $\subset$ SLA; error budget = 1 − SLO** with a freeze policy.
6. **Multi-window multi-burn-rate alerting** — fast burn pages, slow burn tickets.
7. **Alert on symptoms, not causes; actionable + urgent only.**
8. **Mitigate before root-cause; blameless postmortems with owned actions.**
9. **Propagate `traceparent`; tail-sample to keep errors + slow traces.**
10. **Instrument at design time** — observability is a first-class requirement, not an afterthought.

Golden rule: instrument with metrics + logs + traces (+ profiles) linked by a trace id, define SLOs with error budgets and a freeze policy, track percentiles (never averages), bound cardinality, and alert on user-facing symptoms via burn rate — then debug top-down metrics $\rightarrow$ traces $\rightarrow$ logs.

Last updated: 2026-06-15 | Topic: Observability | Level: FAANG Senior